… it will be subject to encoding. Notice the difference between the first `myFunc` output and the output filtered with `|raw` (which also serves as a nice demonstration of chaining filters together). Let's render this template with some data:

```
// index.php
print $twig->render('arrow-func.html.twig', [
    'cat' => 'Jess',
    'human' => 'Anna',
]);
$ curl http://localhost:8080/index.php
<ul>
  &lt;li&gt;cat: Jess&lt;/li&gt;
  &lt;li&gt;human: Anna&lt;/li&gt;
</ul>
<ul>
  <li>cat: Jess</li>
  <li>human: Anna</li>
</ul>
```

Using a template-defined function to insert snippets of markup is not recommended, and the previous example merely serves as a demonstration for `|invoke` and arrow functions. The `{% if %}` and `{% for %}` tags provide a good degree of control over iterative rendering, but as your page grows in complexity, it will become necessary to break your templates into related or reusable units (or lose your sanity trying to maintain one enormous template file… your choice).

Twig `include()`

Twig has a very simple mechanism for stitching templates together: the `include()` function. This works like PHP's `include` language construct: at the point it is encountered, the target file is parsed and rendered before control passes back to the calling template. `include()` accepts a variable number of arguments; in order, these are the relative filepath of the target file (required, relative to the path(s) defined for `FilesystemLoader`), an optional mapping of variables and values for use in the target template, and the remainder are named boolean parameters: